Data Creators › Day 4 — run sheet

Day 4 run sheet — Bitemporal loading and Airflow orchestration

Instructor-only companion to the anchor notebook, in the notebook's order: the same numbered sections and labelled parts, each with its time, purpose, what is on screen, what to say, what to ask at the whiteboard (with the answers rooms give and what to add), the demo steps with their coaching, fallbacks, the transition and what to do if ahead or behind. Keep the notebook on the projector and this beside it. Times follow the instructor plan; adapt them to the room and keep the sequence.

Before the day

- Before the day: `bash demos/airflow_local/run.sh start` (Docker Desktop running; the image is already built) and `bash demos/bitemporal/demo.sh` once. The Airflow UI is http://localhost:8080.

© 2026 Decoded Limited · decoded.com

Sections

#	Section	Time	What happens
1	Bitemporal loading	09:00–09:45	Demo: two clocks, three loads, four questions · Guest or volunteer: a real table · Shareback: which clock?
2	Airflow orchestration	09:45–13:30	Orientation: the DAG, and Friday · Break · Demo: the stack, a run, the UI, a controlled failure · Challenge 10: Airflow practical · Lunch · Shareback: one run, one log, one output check
3	CI/CD and release operations	13:30–15:00	Demo: GitOps, Harness, Datadog · Break · Activity 10: release decision · Activity 11: explain a failure and repair · Friday preparation

Timeline

§	Time	Block	Type	On screen
1	09:00–09:45	Bitemporal loading	Demo + shareback	Anchor › Bitemporal (diagram, demo cells, the macro); a guest or volunteer’s screen
2	09:45–10:00	Orchestration and capstone orientation	Demo	Anchor › The DAG diagram; Friday preview
2	10:00–10:15	Break	Break	

© 2026 Decoded Limited · decoded.com

§	Time	Block	Type	On screen
2	10:15–10:45	Airflow demonstration	Demo	Anchor › Airflow cells; the Airflow UI at localhost:8080
2	10:45–12:00	Challenge 10 — Airflow practical	Challenge	Anchor › Challenge box; learners in challenges/v1/03-airflow-orchestration
2	12:00–13:00	Lunch	Break	
2	13:00–13:30	Continue Airflow practical, regroup at 13:30	Challenge + shareback	Anchor › Challenge box
3	13:30–14:00	CI/CD and deployment context	Demo	Anchor › CI/CD (diagram), Harness walkthrough, Datadog monitors
3	14:00–14:15	Break	Break	
3	14:15–14:35	Release decision in pairs	Challenge + shareback	Anchor › Activity 10 card
3	14:35–14:50	Explain a failure and repair	Challenge + shareback	Anchor › Activity 11 card and template
3	14:50–15:00	Friday preparation and closing questions	Shareback	Anchor › Friday checklist

© 2026 Decoded Limited · decoded.com

1. Bitemporal loading

Demo — two clocks, three loads, four questions · 09:00–09:45

Purpose. Business time and system time; loading strategies; a worked history with a correction; which clock answers which question.

On screen. Anchor › Bitemporal (diagram, demo cells, the macro); a guest or volunteer's screen

Instructor notes.

- A volunteer or guest shows a real table for ten minutes if one was recruited yesterday; the anchor's demo on DuckDB covers the same ground either way.
- Ask the Monday/Wednesday question before opening any code; use one concrete timeline.
- Shareback: each table answers one 'which clock?' question.

Say.

- "The source corrects Monday's value on Wednesday. What was true on Monday, and what did we believe on Tuesday?"

Ask at the whiteboard — loading strategies and two clocks. Ask in order, write what the room says, add what is missing.

Ask	Expect, then add
How can new data land in a table?	Truncate and reload; insert-overwrite; SCD type 2 (effective start and end dates keep history). Bitemporal is SCD2 with a second clock.
What are the two clocks?	Business time: when the fact was true in the world (the business date). System time: when the warehouse loaded or believed it. Separate columns for start and end of each, plus a current indicator and a row hash.

© 2026 Decoded Limited · decoded.com

Ask	Expect, then add
Full, incremental, balance?	Full: the source sends the whole book of record each run (expensive: every row re-versioned). Incremental: only new or changed records with a business key; a known key with new content ends the previous version. Balance: for monetary balances. Some models need a custom strategy.
Why does a correction matter?	Because ‘as was’ must still answer what the report said on Tuesday, and ‘as of’ must give the corrected Monday value today. Overwriting destroys the first; a status flag cannot express either.
UTC or local?	The platform stamps system time in UTC; a run at 03:00 Pacific for the previous business day shows yesterday’s business date. Always say which clock a date column is.

Demo — three loads on DuckDB, then the four questions. Three loads of a tiny associate feed, then the four questions, from one script.

1. Run the cell: `demos/bitemporal/demo.sh` seeds, loads day 1, loads day 2, loads the correction, and prints the history and the as-is, as-of and as-was queries.
2. Read the history row by row: which load wrote each row, and which clock closed it.
3. Run the two `sql()` cells: the audit trail for one associate, then as-of 2 November.
4. Open `demos/bitemporal/macros/bitemporal_load.sql` (the cell prints it): three steps on one screen.
5. Say: the paved path’s `bitemporal_load_v2` does the same job with more strategies.

Guest or volunteer — a real bitemporal table · 09:00–09:45

Shareback — which clock? · 09:00–09:45

Transition. From history to timing: orchestration.

If ahead or behind. Over time: skip the macro walk, keep the four queries.

© 2026 Decoded Limited · decoded.com

2. Airflow orchestration

Orientation — the DAG, and Friday's preview · 09:45–10:00

Purpose. What orchestration is, connected to the pipeline generated on Day 2; a preview of Friday's work and its starting files.

On screen. Anchor › The DAG diagram; Friday preview

Instructor notes.

- Open the eod_prices DAG from Day 2: it is the same shape as the challenge's.
- Friday: Phase 1 scope, two to three hours, the working folder rule.

Say.

- "Airflow plays no instrument. It tells dbt when to play, and what to do when it goes wrong."

Say.

- "Friday: you get CSV files about a fictional wealth manager called Atlas and a README. Phase 1 is the certification bar: sources, the three layers, tests, and understanding the orchestration. Two to three hours for most teams. Phase 2 is fourteen extensions you choose from. We watch you build it; nobody hands out solutions."
- "Work in data_products/atlas_elt, not in the challenge folder. If you run it in the wrong folder, that is on you, and now you have been told."

Transition. Break, then the Airflow demo.

> ☕ Break · 15 min · back at 10:15

Demo — the stack, a run, the UI, a controlled failure · 10:15–10:45

Purpose. Operators, dependencies, callbacks, the UI, Astro blueprints, and a controlled failure, on a local Astro stack running Day 3's project.

On screen. Anchor › Airflow cells; the Airflow UI at localhost:8080

© 2026 Decoded Limited · decoded.com

Instructor notes.

- Start the stack before the day (bash run.sh start); seed; trigger; show graph, grid, a task log, the code tab.
- Read >> as ‘and then’. Open callbacks.py. Show a blueprint and its YAML.
- Controlled failure: stale, trigger, the freshness task fails and the rest is upstream_failed; fresh, trigger, green.

Say.

- “A green graph is not evidence that the data is right. The publish task shows the rows; that is evidence.”

Demo — the stack, a run, the UI. Trigger a run on the local Astro stack and read it in the UI the way learners will.

1. Before the day, in `demos/airflow_local/`: `bash run.sh start`.
2. Run the cells below: seed, trigger, wait, status.
3. Open http://localhost:8080: the DAGs list with owner, schedule and tags.
4. Graph view: five boxes and the dbt task group.
5. Grid view: one column per run.
6. Click `publish_to_catalog` and open its log: the published rows are in it.
7. Code tab: the DAG file as deployed.
8. Say: learners use the same commands: `astro dev run dags list`, `dags trigger`, `tasks states-for-dag-run`.

Demo — a controlled failure, then the repair. Break the freshness gate on purpose, read the log, repair, rerun.

1. Run the stale cell: the custodian feed looks two days old; a run is triggered.
2. Watch the freshness gate fail and everything downstream go `upstream_failed`: no publish, no rows.
3. Open the failed task’s log in the UI and read the first useful line aloud.
4. Run the fresh cell: the feed is stamped fresh, a run is triggered, green.
5. Say: this is the same drill as the challenge’s Phase 2 and Friday’s questions: late input, failed check, repeated run.

Transition. Challenge 10 until lunch.

If ahead or behind. If the stack misbehaves, the status and log cells still show a previous green run.

© 2026 Decoded Limited · decoded.com

Challenge 10 — Airflow practical · 10:45–12:00

Purpose. Launch the assigned Airflow project; learners wire the chain, run tasks inside the scheduler, inspect states and logs.

On screen. Anchor › Challenge box; learners in challenges/v1/03-airflow-orchestration

Instructor notes.

- Boot order matters: setup, then seeds, then astro dev start; the cold-start symptom and fix are in the anchor.
- Collect deployment and monitoring questions for the afternoon.

Say.

- “Which run produced this result, and what proves the data is right?”

Challenge — Airflow practical (challenge 03). `challenges/v1/03-airflow-orchestration/README.md` · until 13:30, with lunch.

Transition. Lunch.

If ahead or behind. Fast finishers: the extension questions (late input, failed check, repeated run), then Phase 2.

> 🍽 Lunch · 60 min · back at 13:00

Shareback — one run, one log line, one output check per pair · 13:00–13:30

Purpose. Resolve blockers; Phase 2 for those on it; regroup with one run, one log and one output check per pair.

On screen. Anchor › Challenge box

Instructor notes.

- Keep credential and path errors separate from the intended data-quality failure.
- Shareback: a pair shows the current run, a useful log line, and the warehouse output.

Say.

- “Show me the run ID, the log line that told you what happened, and the table it produced.”

© 2026 Decoded Limited · decoded.com

Transition. CI/CD and the deployment picture.

© 2026 Decoded Limited · decoded.com

3. CI/CD and release operations

Demo — GitOps, Harness, Datadog · 13:30–14:00

Purpose. Where code goes, how deployment is initiated, where execution is visible, how failures are investigated: Harness, Astro, Datadog.

On screen. Anchor › CI/CD (diagram), Harness walkthrough, Datadog monitors

Instructor notes.

- Agree the agenda with a guest in advance; otherwise the anchor's walkthrough with screenshots.
- Say the downsides of CI/CD too; rooms respect it.

Say.

- "A pull request is where acceptance happens. A deleted test is a block of red lines that is easy to skim past."

Ask at the whiteboard — CI/CD. Ask in order, write what the room says, add what is missing.

Ask	Expect, then add
What do CI and CD mean?	Integration: merging a feature branch, often. Deployment: releasing to an environment, including production, often.
Why bother?	Faster feedback that something is broken; a shared understanding of the product across the team; observability of the lifecycle.
What are the downsides?	Automation without a real human in the loop; fragility; correlated failures (when one shared system is down, everything is); the maintenance, approval and audit burden. It is not free.
What is Harness here?	The CI/CD system, roughly GitOps: a pull request submitted and approved triggers the pipeline. It isolates the creator from the deployment machinery.

© 2026 Decoded Limited · decoded.com

Demo — Harness, with screenshots or a guest. Walk a deployment to SQA and then to production: live if someone has access, else from the pre-flight screenshots.

1. Pick the environment and find the repository's pipeline.
2. SQA: Run, apply the SQA input set, choose the feature branch, run; a few minutes.
3. Check who deployed to SQA last and whether they finished testing: SQA is shared.
4. Production: the PRD pipeline, fixed values, the release branch, and a change record (CRQ) number before anything runs.
5. Open the logs of a failed execution.
6. Say: secrets live in AWS Secrets Manager, never in code or a chat; an API token that lets an AI assistant trigger deployments is a risk, not a shortcut.

Transition. Break, then the release decision.

> ☕ Break · 15 min · back at 14:15

Activity 10 — release decision · 14:15–14:35

Purpose. A failed candidate with 500 rows and 497 distinct IDs, yesterday's accepted output available: decide what ships and what to tell the consumer.

On screen. Anchor › Activity 10 card

Instructor notes.

- Ten minutes deciding and drafting, ten discussing. The failed candidate stays blocked; whether yesterday's output is acceptable depends on the consumer's freshness commitment.

Say.

- "What information is missing, and who can supply it?"

Activity 10 — Release decision. Pairs, 20 min; the card is in `activities/`.

Transition. Write the failure note.

Activity 11 — explain a failure and repair · 14:35–14:50

Purpose. A short note another creator could use to locate the run and understand what happened; swapped and tested.

© 2026 Decoded Limited · decoded.com

On screen. Anchor › Activity 11 card and template

Instructor notes.

- If a pair’s execution was blocked, they use the anchor’s prepared failure example and separate expected from observed.

Say.

- “Could another creator find this run from your note alone?”

Activity 11 — Explain a failure and repair. Pairs, then swap, 15 min; the card is in `activities/`.

Transition. Friday preparation.

Friday preparation — the checklist, then the close · 14:50–15:00

Purpose. Check the starting files and environment; record remaining problems and who resolves them.

On screen. Anchor › Friday checklist

Instructor notes.

- Every learner opens the capstone README and runs the readiness checks; problems go on the list with a name.

Say.

- “Tomorrow starts sharp at nine. The sharper you start, the sharper you finish.”

Before learners leave

- [ ] The DAG, run and task identities, useful logs, an output check and a short explanation of the failure and repair, per pair.
- [ ] Every learner has opened challenges/v1/04-atlas-capstone/README.md and run the readiness checks in data_products/atlas_elt.
- [ ] A list of setup issues that could stop Friday’s start, each with the person responsible for the next action.
- [ ] The Airflow stack stopped on the learners’ laptops (astro dev stop) so Friday starts clean.

© 2026 Decoded Limited · decoded.com